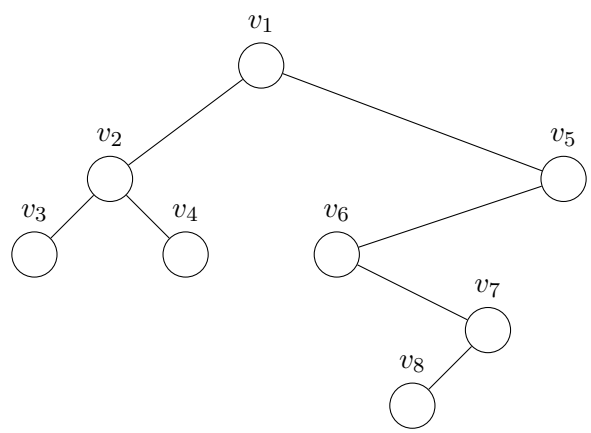
Introduction

Definition 1. A **binary tree** is a tree (a connected acyclic graph) with the following properties

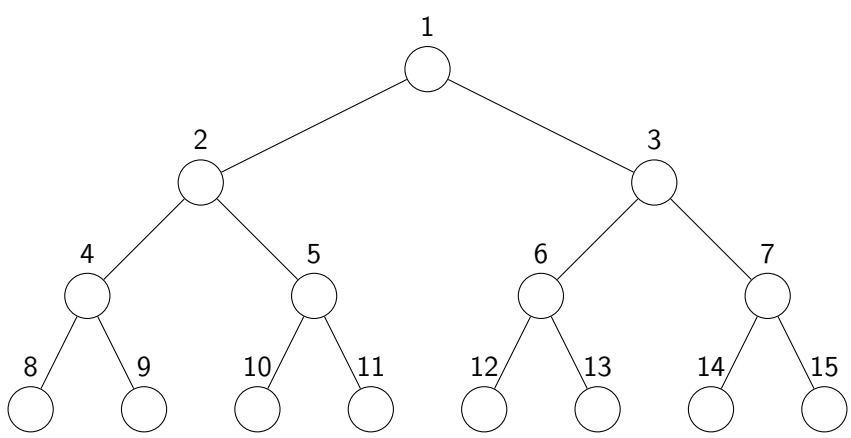
1. Every node has zero, one or two children.
2. One node is the root.
3. Every node, except the root, has one parent.
4. A node with no children is called a leaf.

Definition 2. The **height** of a binary tree is the number of edges on the longest path from the root to a leaf.



Definition 3. A **complete binary tree** is a binary tree where:

1. All the internal nodes have EXACTLY two children
2. All the leaves are at the same distance from the root.



We shall represent a binary tree as a collection binary tree nodes with only node stored directly, the so-called "root" of the tree. Each node will contain the following elements

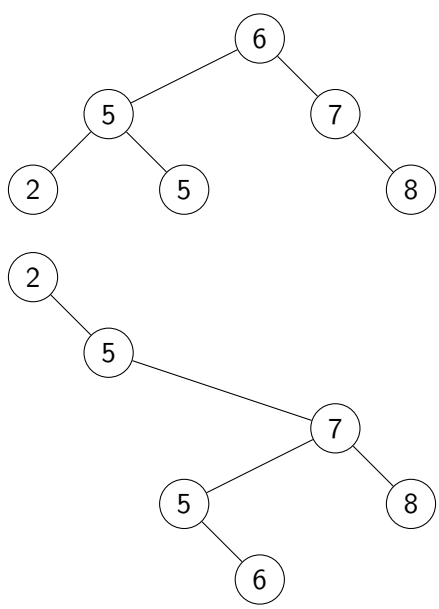
Variable	Type	Represents
`left`	Pointer	A pointer to the left child of the node
`right`	Pointer	A pointer to the right child of the node
`parent`	Pointer	A pointer to the parent of the node
`key`	Numerical	The value stored at the node

The root is distinguished as the only node whose parent is **NIL**.

Definition 4 (Binary Search Tree Property)**.** A binary tree T is a binary search tree if: let node y be a descendant of node x.

1. If y is in the left subtree of x, then $y.\texttt{key} \leq x.\texttt{key}$
2. If y is in the right subtree of x, then $y.\texttt{key} \geq x.\texttt{key}$.

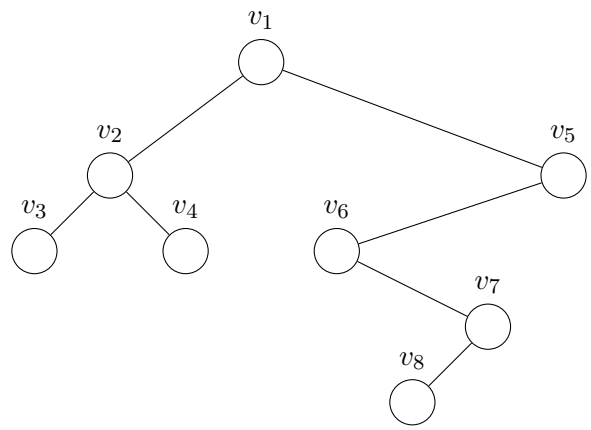
There are potentially many valid binary search trees for a single set of values; such as,

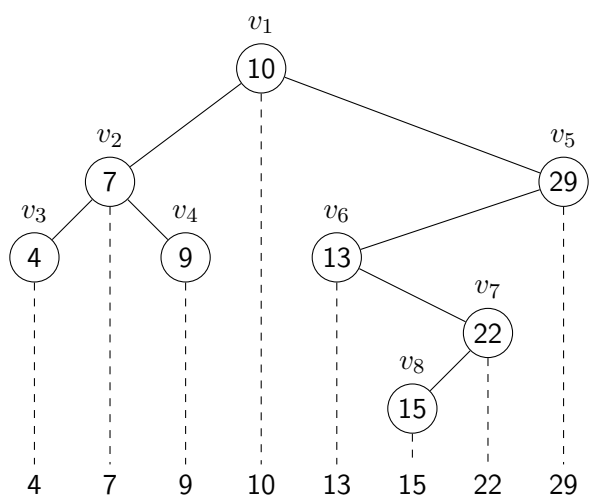


Example 1. Assign the values

$$4, 7, 9, 10, 13, 15, 22, 29$$

to the binary tree and make it a binary search tree.





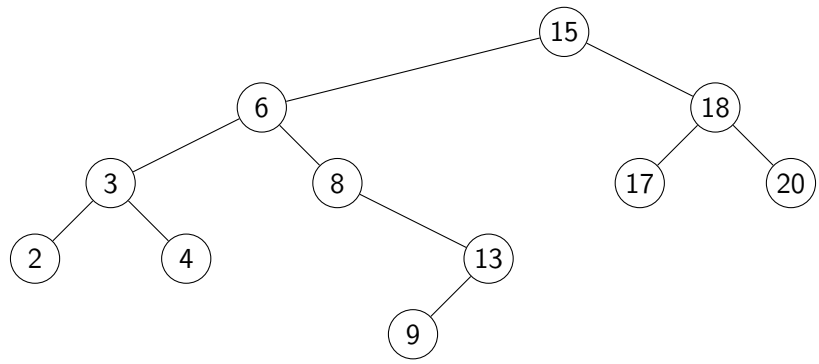
Notice that we have simply assigned the values from left to right in ascending order!

We can perform a "walk" on a tree such that we visit every node in the tree exactly once. One walk is the "in-order" walk that will print the nodes of the binary search tree in sorted order

```
function InorderTreeWalk(x)
    if x ≠ NIL then
        InorderTreeWalk(x.left)
        Print(x.key)
        InorderTreeWalk(x.right)
    end if
end function
```

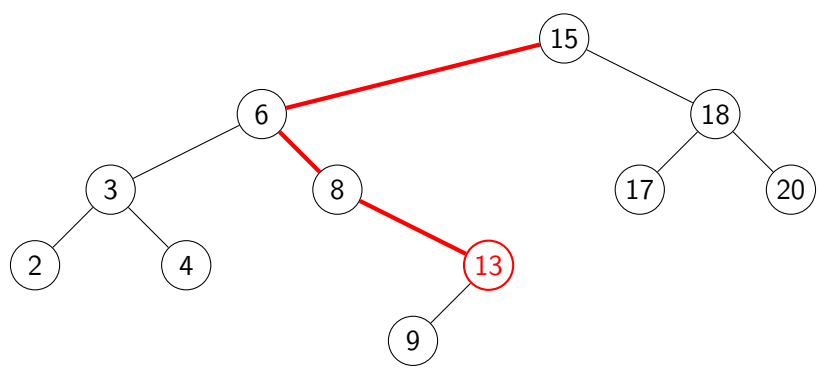
This algorithm takes $\Theta(n)$ time since it visits every node once and only once.

As the name may suggest a binary search tree can be searched efficiently. For example, if we want to find the value 13 in the tree



we make the following comparisons:

1. Since $13 < 15$, 13 must be in the left subtree of 15.
2. Since $13 > 6$, 13 must be in the right subtree of 6.
3. Since $13 > 8$, 13 must be in the right subtree of 8.
4. $13 = 13$. Hooray!



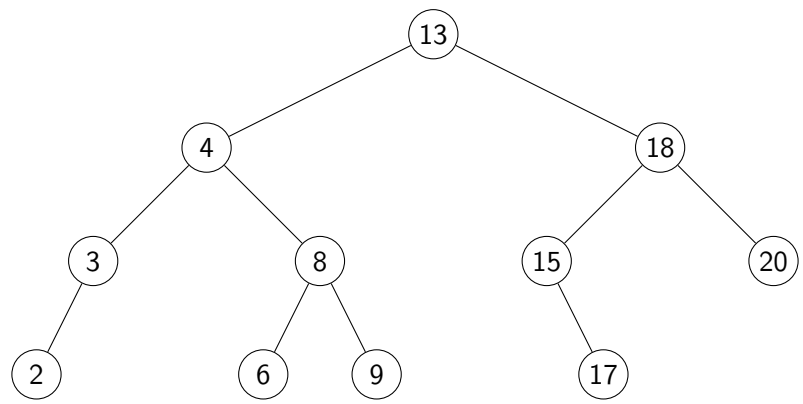

```
function TreeSearch(x,K)
    if (x = NIL) or K = x.key then
        return x
    else if K < x.key then
        return TreeSearch(x.left,K)
    else
        return TreeSearch(x.right,K)
    end if
end function
```

Sine the algorithm always descends at most a single level in the tree, the running time is $O(h)$, where h is the height of the tree.

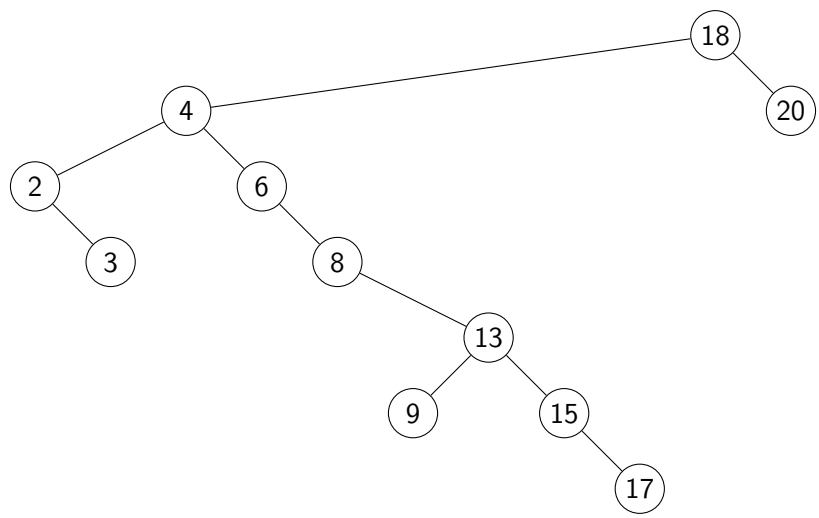
We can also implement the algorithm using a loop structure instead of recursion. The running time remains the same.

```
function IterativeTreeSearch(x,K)
    while (x ≠ NIL) and (K ≠ x.key) do
        if K ≤ x.key then
            x = x.left
        else
            x = x.right
        end if
    end while
    return x
end function
```

Some trees are "balanced"



and others are not (i.e. they are "unbalanced")

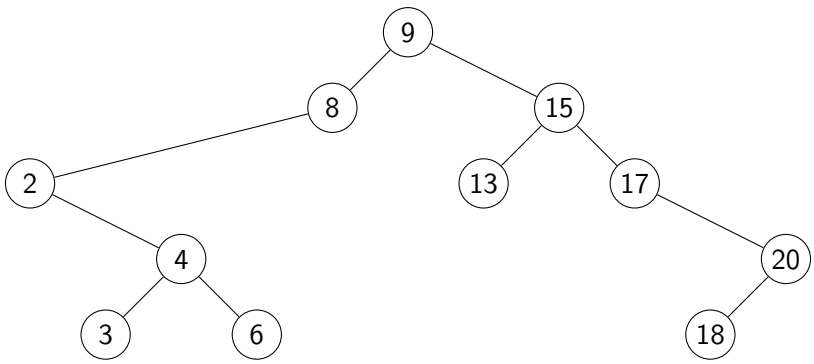


Determining whether a tree is balanced and attempting to "balance" a tree can be a difficulty process. We will discuss this idea more when we discuss tree rotations and red-black trees; for now we simply mention that the height of a binary search tree is bounded by

$$\log_2(n+1) - 1 \leq h \leq n - 1$$

Methods

Suppose we want to find the minimum value of a binary search tree. Let x be the leftmost value in the tree. By definition x has no left subtree. Any values in x's right subtree are necessarily at least as large as x by definition. If p is the parent of x (or any ancestor for that matter), then $x.\texttt{key} \leq p.\texttt{key}$ by definition. Furthermore, if z is in the right subtree of p (or any ancestor of x), then $x.\texttt{key} \leq p.\texttt{key} \leq z.\texttt{key}$. Therefore, the leftmost value is the minimum value in the binary search tree.




```
function TreeMin(x)
    while x.left ≠ NIL do
        x = x.left
    end while
    return x
end function
```

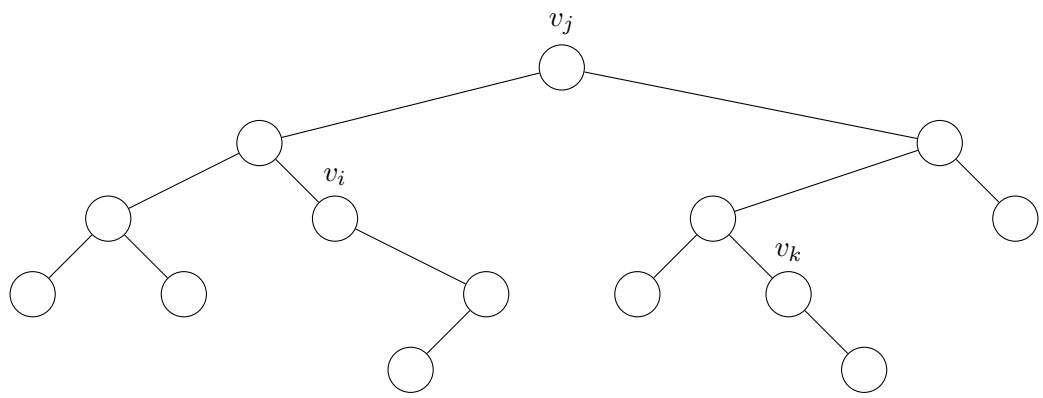
At worst this method must find the leaf that is furthest from the root; in that case the algorithm takes $O(h)$ time.

We can make a similar argument that the maximum element is the rightmost element in the binary search tree.

```
function TreeMax(x)
    while x.right ≠ NIL do
        x = x.right
    end while
    return x
end function
```

And the running time of this method (as with `TreeMin`) is $O(h)$.

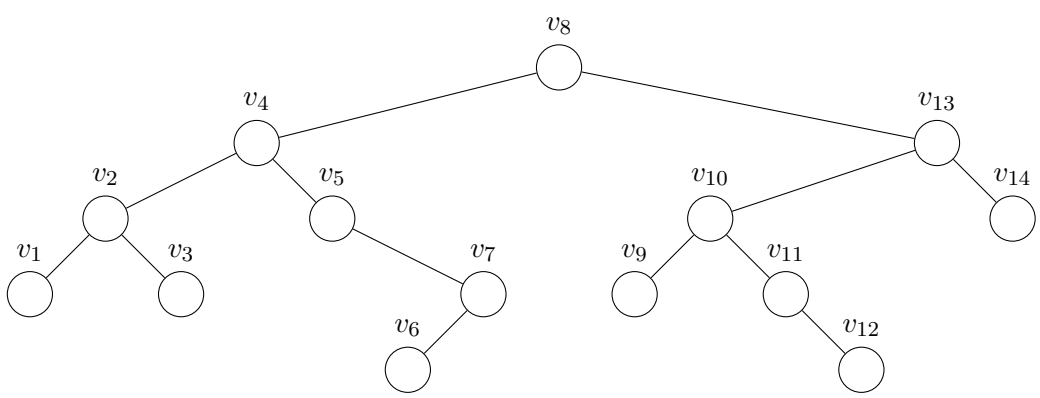
We now will introduce the idea of the "in-order" sequence for a binary search tree. This is an ordering of the nodes $v_1, v_2, v_3, \ldots, v_n$ such that $v_1.\texttt{key} \leq v_2.\texttt{key} \leq \cdots \leq v_n.\texttt{key}$.



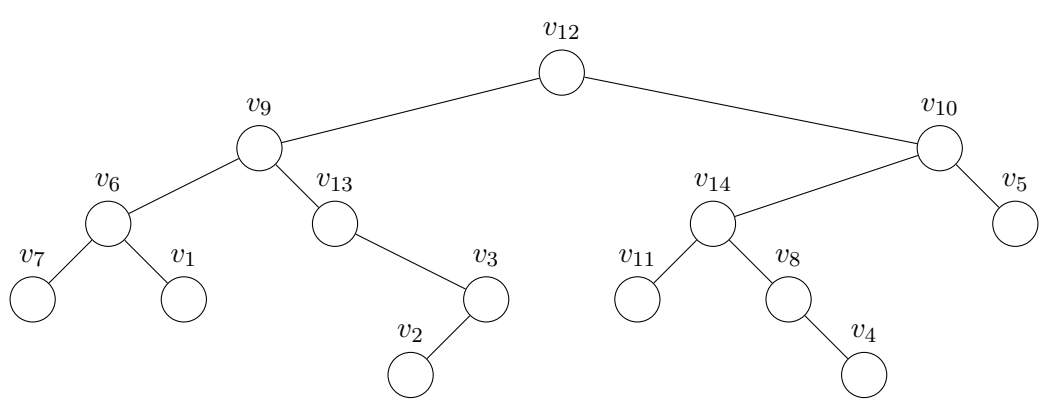
1. If node v_i is in the subtree rooted at $v_j.\texttt{left}$, then $v_i \preccurlyeq v_j$
2. If node v_k is in the subtree rooted at $v_j.\texttt{right}$, then $v_j \preccurlyeq v_k$

Note: If node $v_i$is in the subtree rooted at $v_j.\texttt{left}$ and node v_k is in the subtree rooted at $v_j.\texttt{right}$, then $v_i \preccurlyeq v_j \preccurlyeq v_k$ (i..e $\preccurlyeq$ is a transitive binary relation).

Question: What is the in order sequence of vertices for the following trees?



$v_1 \preccurlyeq v_2 \preccurlyeq v_3 \preccurlyeq v_4 \preccurlyeq v_5 \preccurlyeq v_6 \preccurlyeq v_7 \preccurlyeq v_8 \preccurlyeq v_9 \preccurlyeq v_{10} \preccurlyeq v_{11} \preccurlyeq v_{12} \preccurlyeq v_{13} \preccurlyeq v_{14}$

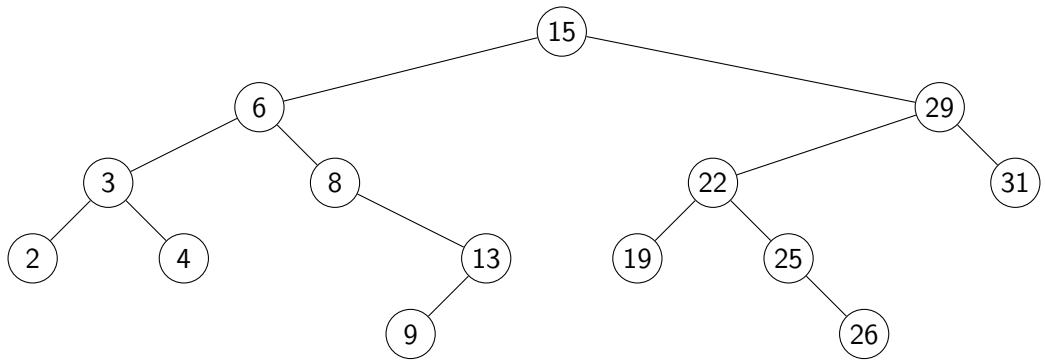


$v_7 \preccurlyeq v_6 \preccurlyeq v_1 \preccurlyeq v_9 \preccurlyeq v_{13} \preccurlyeq v_2 \preccurlyeq v_3 \preccurlyeq v_{12} \preccurlyeq v_{11} \preccurlyeq v_{14} \preccurlyeq v_8 \preccurlyeq v_4 \preccurlyeq v_{10} \preccurlyeq v_5$

Definition 5. The **successor** of node v_i is the next node in the inorder sequence.

Definition 6. The **predecessor** of node v_i is the previous node in the inorder sequence.

Observation: If T is a binary search tree and all the values at nodes are different, then the successor of node v_i is the node with smallest value greater than the value of v_i.



Some observations:

1. The successor of 6 is 8 (its right child)
2. The successor of 13 is 15 (its great grandparent)
3. The successor of 15 is 19 (the ???????)

After much consideration we observe the following cases for finding the successor:

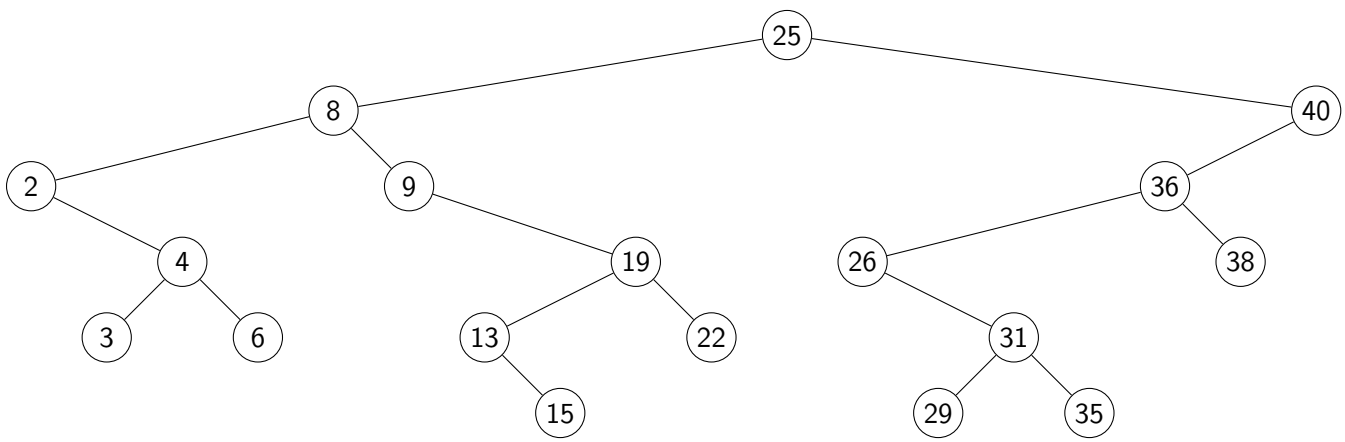
Case I If x.`right` $\neq$ **NIL**, return `TreeMin`(x.`right`)

Case II If x.`right` $=$ **NIL**, return closest ancestor y of x where x is in the left subtree of y

```
function TreeSuccessor(x)
    if x.right ≠ NIL then
        return TreeMin(x.right)
    end if
    y = x.parent
    while (y ≠ NIL) and (x = y.right) do
        x = y
        y = y.parent
    end while
    return y
end function
```

This method either finds the TreeMin (which takes $O(h)$ time) or will (in the worst case) ascend the entire tree which takes $O(h)$ time.

Suppose we want to report all nodes of the following tree with keys in the range $[8, 26]$

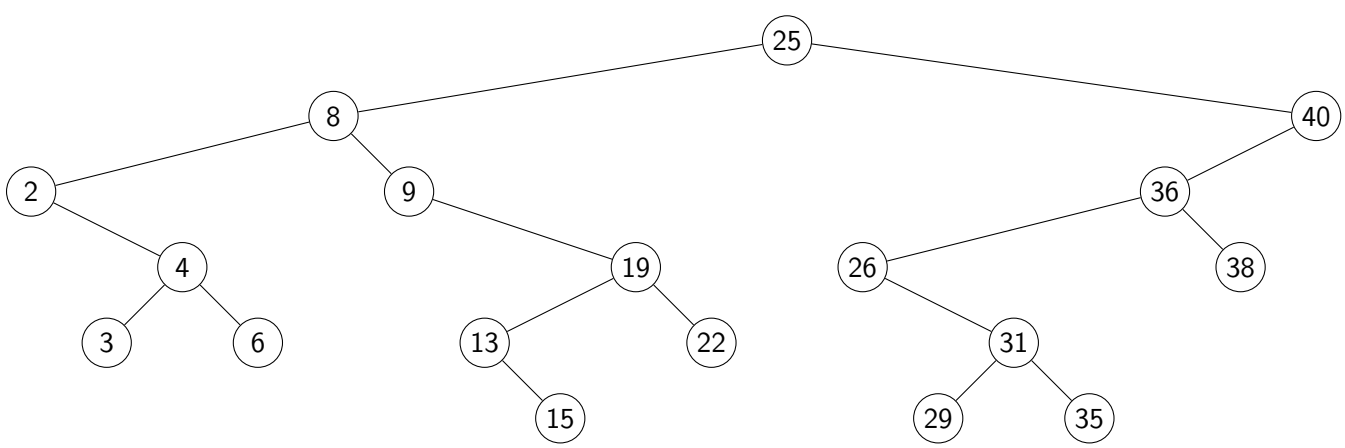



```
function TreeRangeReport(x,k_min,k_max)
    if x ≠ NIL then
        if k_min ≤ x.key then
            TreeRangeReport(x.left,k_min,k_max)
        end if
        if k_min ≤ x.key ≤ k_max then
            Print(x.key)
        end if
        if x.key ≤ k_max then
            TreeRangeReport(x.right,k_min,k_max)
        end if
    end if
end function
```

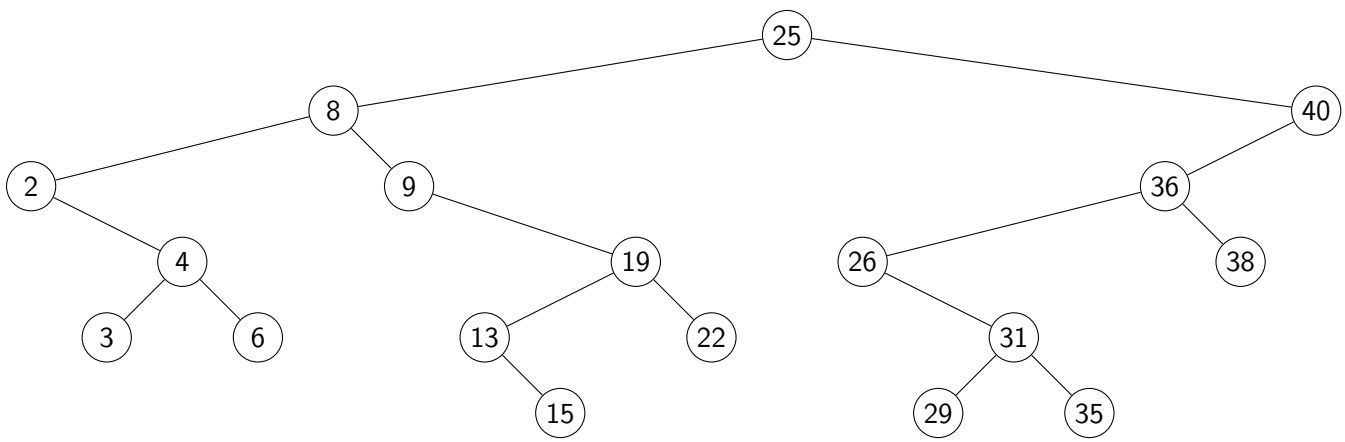
Note that this algorithms runtime depends upon the number of nodes within the given range of values. Let I =number of nodes between k_{min} and k_{max}. We observe that the code must always report every node in the range *and* it will always search until it finds a leaf node of the tree. Therefore, the running time of this method is $O(h + I)$.

```
function TreeMinGE(T,K)
    ▷ Return node with minimum key greater than or equal to K
    currentBest = NIL
    v = T.root
    while v ≠ NIL do
        if v.key ≥ K then
            currentBest = v
            v = v.left
        else
            v = v.right
        end if
    end while
    return currentBest
end function
```

This algorithm is effectively idenitcal to search which we have already shown takes $O(h)$ time.



What if we only wish to know *how many* nodes lie within a range of values?



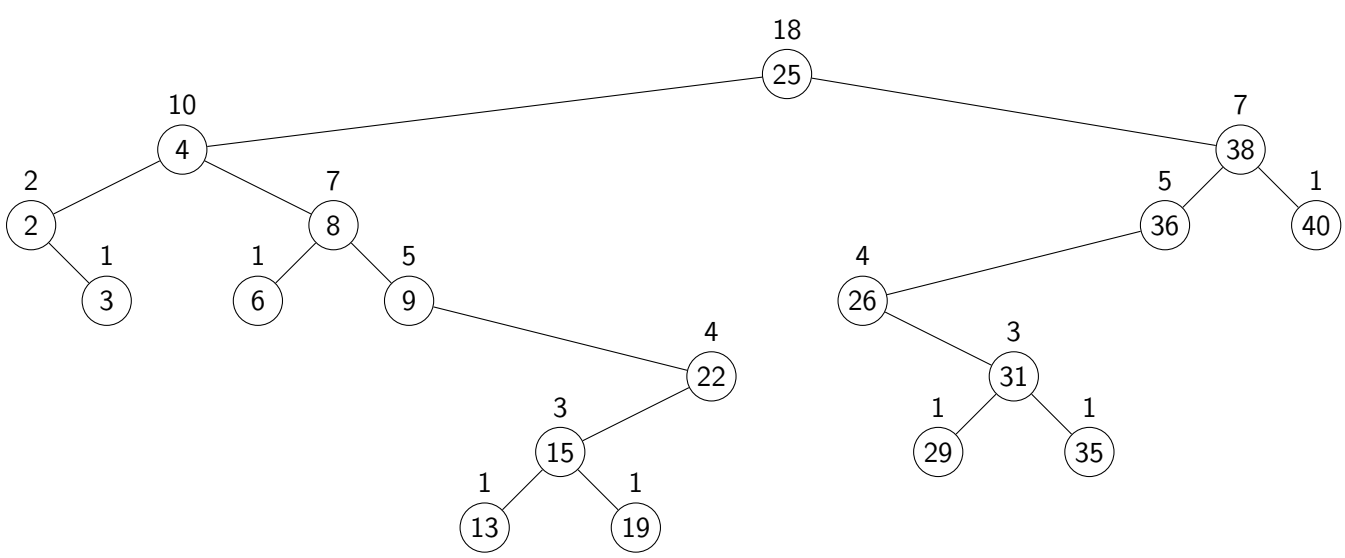

```
function TreeRangeCount(x, k_min, k_max)
    count = 0
    if x ≠ NIL then
        if k_min ≤ x.key then
            count = count + TreeRangeCount(x.left, k_min, k_max)
        end if
        if k_min ≤ x.key ≤ k_max then
            count = count + 1
        end if
        if x.key ≤ k_max then
            count = count + TreeRangeCount(x.right, k_min, k_max)
        end if
    end if
    return count
end function
```

This method is unfortunate since it effectively replicates the method `TreeRangeReport` even though it does not necessitate visiting every node (since we only care about how many nodes are within the range of values, not which nodes).

Idea: For each node u in the binary search tree, add a field u.`size` which stores the number of nodes in the subtree rooted at u.

```
function TreeComputeSize(x)
    TreeComputeSize(x.left)
    TreeComputeSize(x.right)
    x.size = 1
    if x.left ≠ NIL then
        x.size = x.size + x.left.size
    end if
    if x.right ≠ NIL then
        x.size = x.size + x.right.size
    end if
end function
```

Compute the size for each node in the tree given below.



Count the number of nodes greater than or equal to 6.

```
function TreeCountGE(x, K)
    count = 0
    v = x
    while v ≠ NIL do
        if v.key ≥ K then
            count = count + 1
            if v.right ≠ NIL then
                count = count + x.right.size
            end if
            v = v.left
        else
            v = v.right
        end if
    end while
end function
```

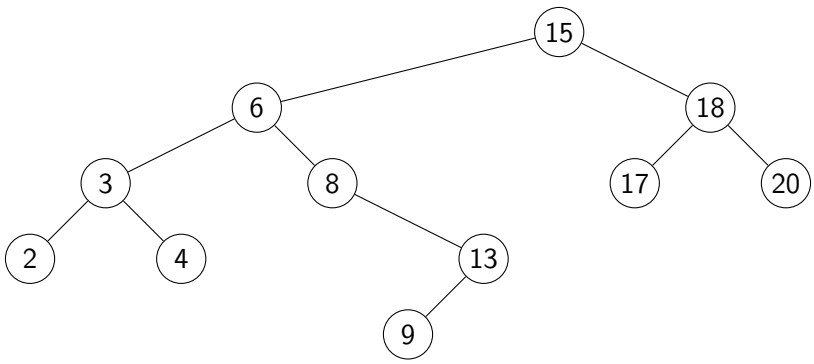
```
function TreeCountLE(x, K)
    count = 0
    v = x
    while v ≠ NIL do
        if v.key ≤ K then
            count = count + 1
            if v.left ≠ NIL then
                count = count + x.left.size
            end if
            v = v.right
        else
            v = v.left
        end if
    end while
end function
```

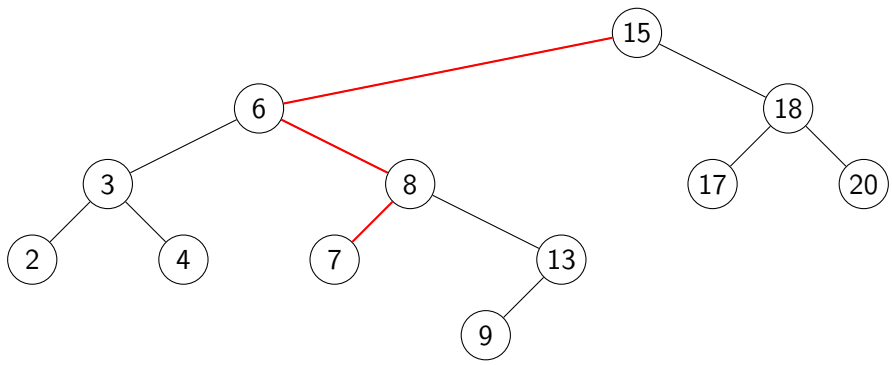
Both of these methods always descend the tree by one level, therefore the running time is in $O(h)$ time.

```
1: function TreeRangeCount(x,k_min,k_max)
2:     v = x
3:     while (v ≠ NIL) and (x ∉ [k_min, k_max]) do
4:         if v.key ≤ k_min then
5:             v = v.right
6:         else if v.key ≥ k_max then
7:             v = v.left
8:         end if
9:     end while
10:    if v ≠ NIL then
11:        count_L = TreeCountGE(v.left, k_min)
12:        count_R = TreeCountLE(v.right, k_max)
13:        return 1 + count_L + count_R
14:    else
15:        return 0
16:    end if
17: end function
```

The while loop takes $O(h)$ time since it always descends the tree by one level and the if-statement on lines 10-16 takes $O(h)$ time because the two methods called are both in $O(h)$. Therefore, the running time is $O(h)$.

We shall now discuss how to insert a new element into a binary search tree. This method is fairly straightforward; search for a valid location for the element and then insert it. Suppose that we want to insert 7 into the binary search tree give below





We abstract the notion of "finding the parent" of the new node:

```
function LocateParent(T,z)
  ▷ Return the future parent of z
    y = NIL
    x = T.root
    while x ≠ NIL do
        y = x
        if z.key < x.key then
            x = x.left
        else
            x = x.right
        end if
    end while
    return y
end function
```

Since this method always descends a single level of the binary search tree it takes $O(h)$ time.

To perform an insertion, we shall first find the parent of the new node and then we need only assign all of the pointer values correctly.

```
function TreeInsert(T,z)
    y = LocateParent(T, z)
    z.parent = y
    if y = NIL then
        T.root = z
    else if z.key < y.key then
        y.left = z
    else
        y.right = z
    end if
end function
```

`LocateParent` takes $O(h)$ time and the pointer rearrangement takes constant time, so the running time of this method is $O(h)$.

We can also modify our insertion method to update the size of the tree after we perfom the insertion.

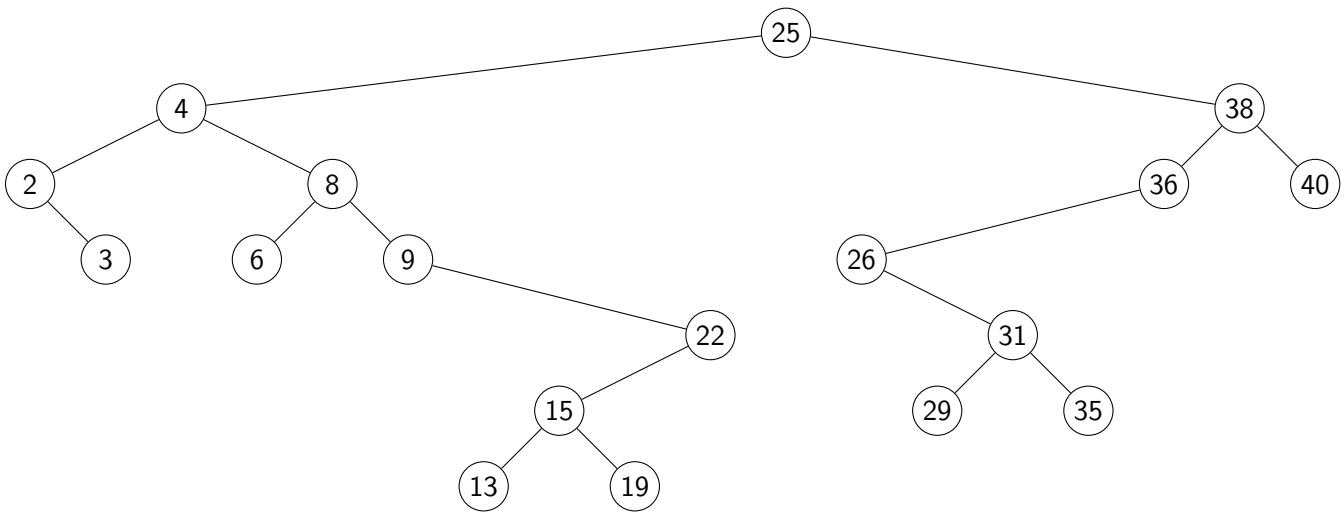
```
function InsertAndUpdateSize(T,z)
    TreeInsert(T,z)
    z.size = 1
    y = z.parent
    while y ≠ NIL do
        y.size = y.size + 1
        y = y.parent
    end while
end function
```

Insertion takes $O(h)$ time and updating the size ascends the tree until the root is found which takes $O(h)$ time as well.

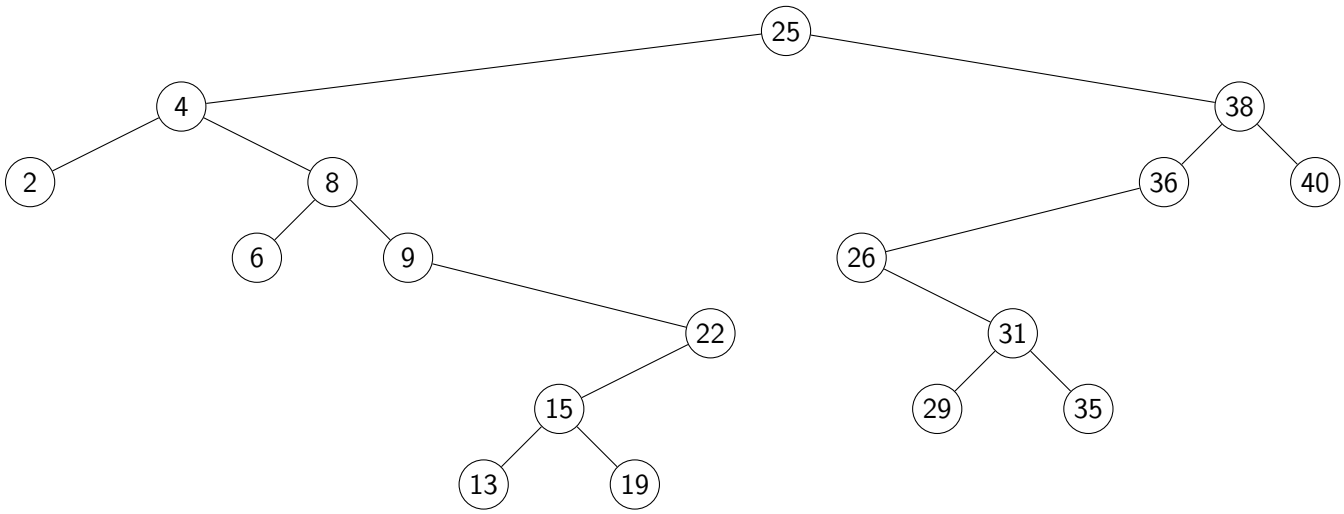
We shall now discuss how we can delete nodes from a binary search tree. We shall accomplish this in the following manner:

1. Find the node to be deleted.
2. Remove the node from the tree and replace it by another node in the tree.

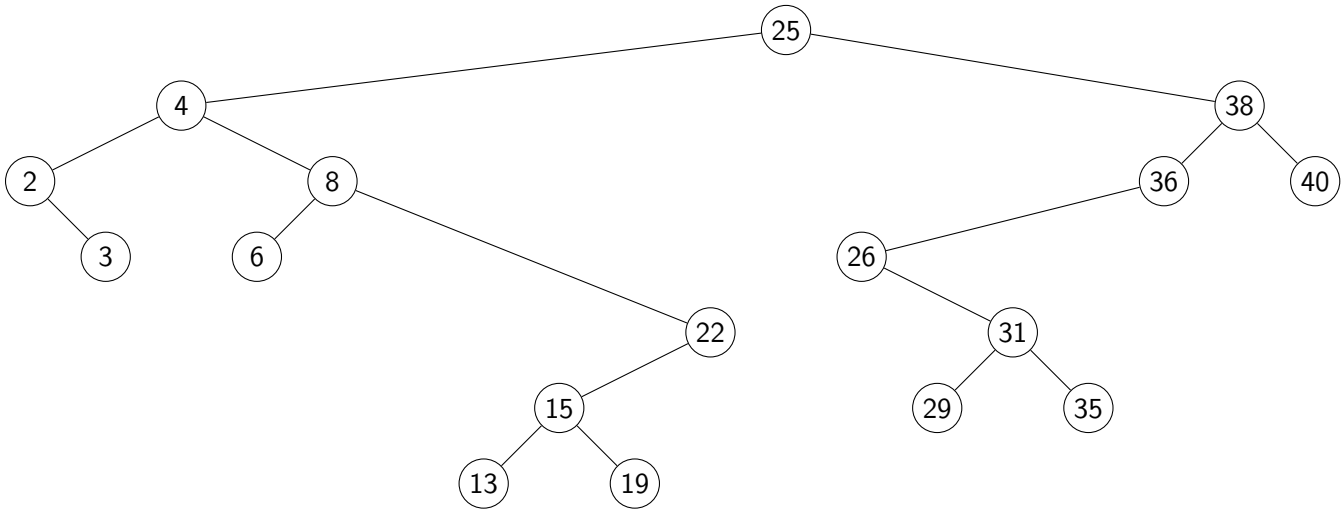
We shall perform several deletions in the following binary search tree (each deletion will assume that this tree is the base).



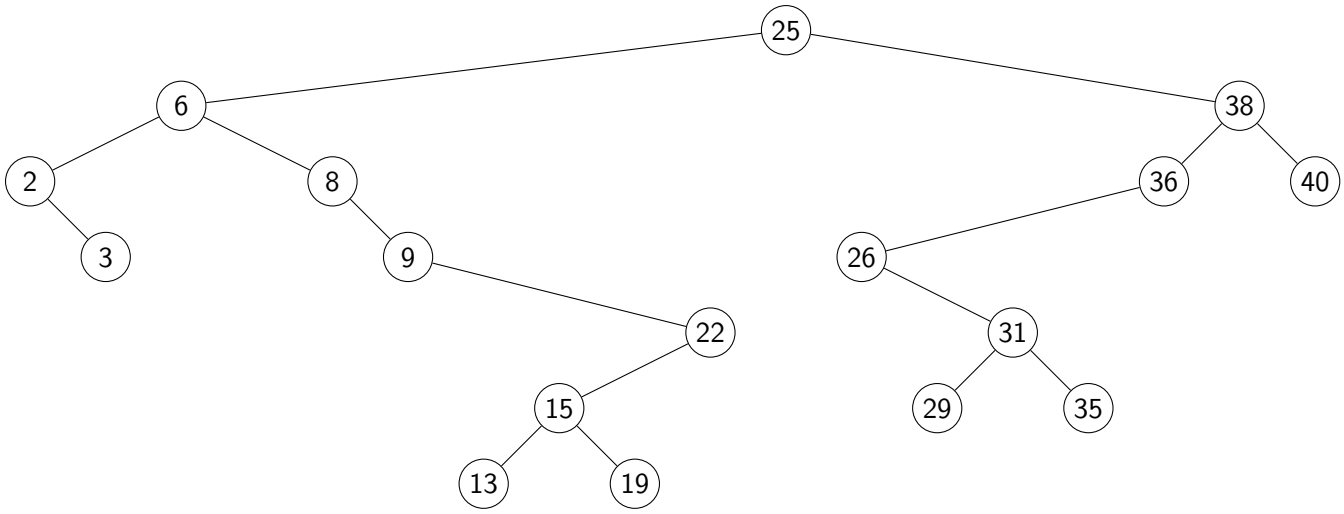
`Delete`(3) – 3 is removed and no other changes occur.



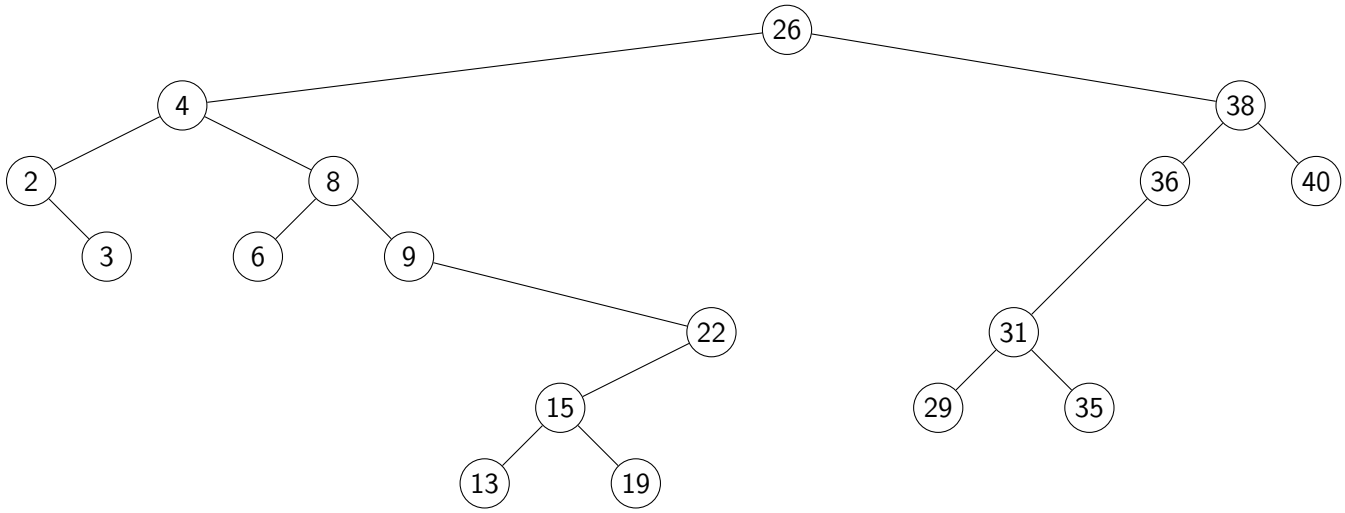
`Delete`(9) – 9 is removed, 9's right child (22) is now the right child of 8.



`Delete`(4) – 4 is removed and replaced by its successor (6).



`Delete`(25) – 25 is removed, it is replaced by its successor (26). The right child of 26 (31) is made the left child of the parent of 26 (36).



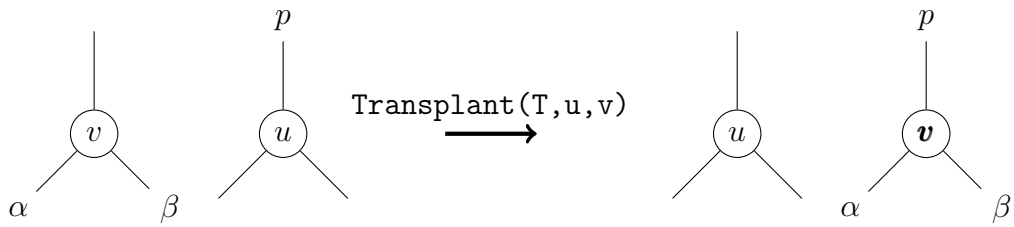
There are three primary cases:

Case I If node z has no children, delete z

Case II If node z has one child, replace z with its child.

Case II If node z has two children, replace z with its successor y and replace y with its right child.

Before we formally introduce the delete method we shall introduce the notion of transplanting a node.



Notice that we take u and replace it with v and v's children.

```
function Transplant(T,u. v)
    p = u.parent
    if p = NIL then
        T.root = v
    else if u = p.left then
        p.left = v
    else
        p.right = v
    end if
    if v ≠ NIL then
        v.parent = p
    end if
end function
```

This code takes $\Theta(1)$ time since all that is done is pointer rearrangement.

The first two clauses of the if-else block deal with the case of 0 children and 1 child. The third clause tackles the case of finding the successor of the node and successfully transplanting the node into the location occupied by z.

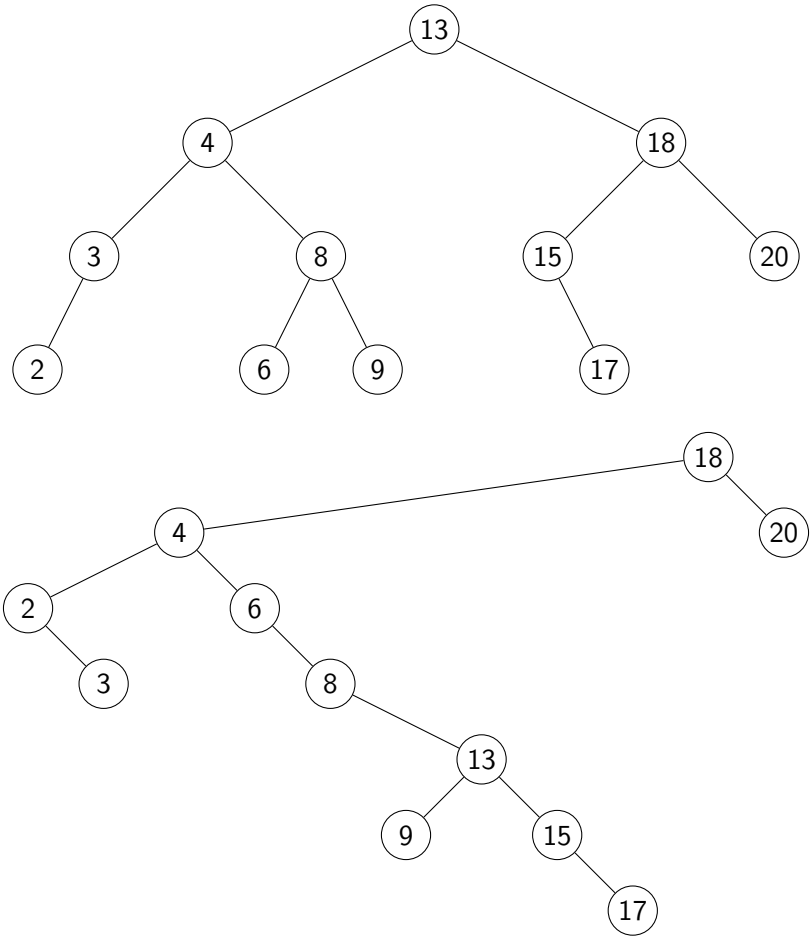
```
1: function TreeDelete(T,z)
2:     if z.left = NIL then
3:         Transplant(T,z,z.right)
4:     else if z.right = NIL then
5:         Transplant(T,z,z.left)
6:     else
7:         y = TreeMin(z.right) ▷ y is the successor of z
8:         if y ≠ z.right then
9:             Transplant(T,y,y.right)
10:            y.right = z.right
11:            y.right.parent = y
12:        end if
13:        Transplant(T,z,y)
14:        y.left = z.left
15:        y.left.parent = y
16:    end if
17: end function
```

Lines 2-6 take constant time; line 7 takes $O(h)$ time; lines 8-16 take constant time (since `Transplant` and pointer rearrangement take constant time). So, deletion takes $O(h)$ time. We can modify this method to work with our size field by updating the size after deletion. We leave it to the reader to analyze the correctness of this algorithm.

```
function UpdateSize(T,z)
    while z ≠ NIL do
        z.size = 1
        if z.left ≠ NIL then
            z.size = z.size + z.left.size
        end if
        if z.right ≠ NIL then
            z.size = z.size + z.right.size
        end if
        z = z.parent
    end while
end function
```

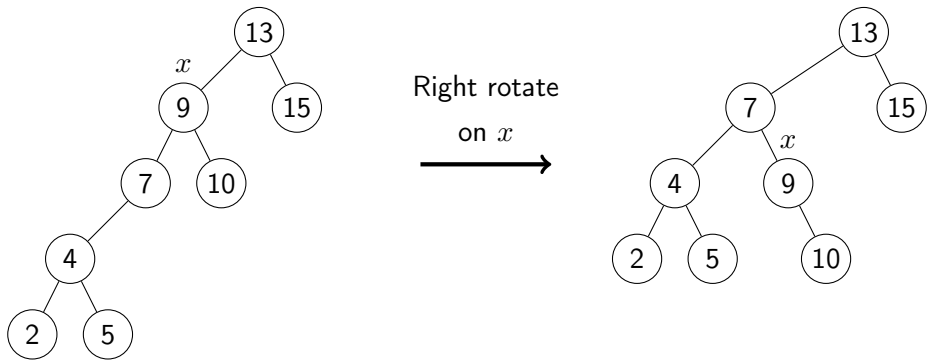
```
function TreeDelete(T,z)
    if z.left = NIL then
        Transplant(T,z,z.right)
        UpdateSize(T,z.parent)
    else if z.right = NIL then
        Transplant(T,z,z.left)
        UpdateSize(T,z.parent)
    else
        y = TreeMin(z.right) ▷ y is the successor of z
        x = y.parent
        if y ≠ z.right then
            Transplant(T,y,y.right)
            y.right = z.right
            y.right.parent = y
        end if
        Transplant(T,z,y)
        y.left = z.left
        y.left.parent = y
        UpdateSize(T,x)
    end if
end function
```

Consider the two binary search trees



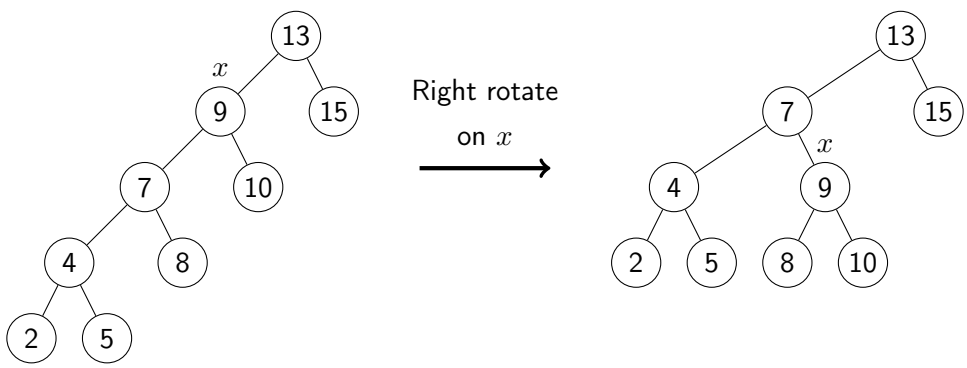
Can we perform any set of operations to transform the unbalanced tree into a more balanced tree? Note that we need to ensure that the binary search tree property is preserved! The operation which solves this problem is called a "rotation". It is a (local) rearrangement of nodes such that their relative positions are changed, but the binary search tree property is maintained. Let us begin by analyzing some simpler trees.

A right rotation on 9 takes the node 9 and moves it "down and to the right". That is, it takes the **left child** (7) of 9 and makes it the parent of 9. In order to ensure the BST property is preserved this means that 9 is now the **right child** of 7.



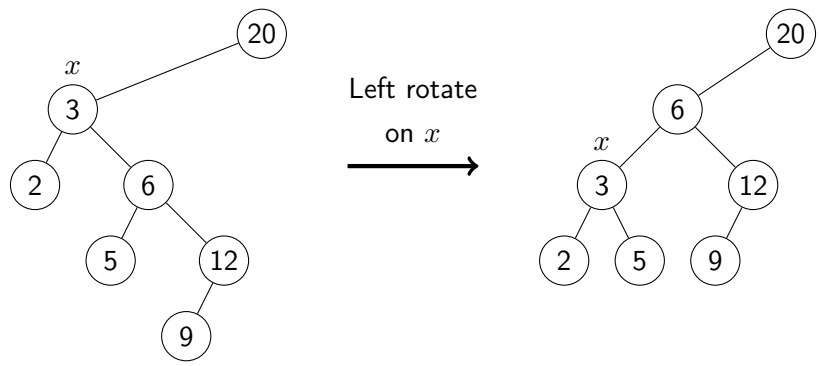
This operation has reduce the height of the tree by 1! The height of the tree on the left is 4 and the height of the tree on the right is 3! Note that it is impossible to reduce the height to 2 since a tree of height 2 has at most $2^3 - 1 = 7$ nodes and our tree has 8.

We modify the tree by introducing a right child to node 7. This poses a problem since we want to make 9 the right child of 7. We can resolve this issue by finding another valid location for that node. Note that $7 \leq 8 \leq 9$ since 7 is in the left subtree of 9 and 8 is in the right subtree of 7. Therefore, since $8 \leq 9$ we can make 8 the left child of 9; we know with certainty that this pointer will point to **NIL** since prior to the rotation it pointed at 7.

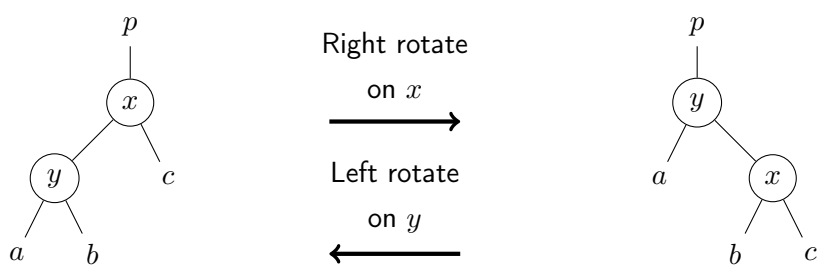


Again, the rotation has reduced the height of the tree by 1.

Since right rotations exist, it stands to reason that we should also define a left rotation. A left rotation on 3 takes the **right child** (6) of 3 and makes it the parent of 3 (with 3 being the left child of 6). Additionally, it takes the left child of 6 and makes it the right child of 3.



As before, this rotation successfully reduces the height of the tree from 4 to 3!
Notice that these methods only modify a handful of nodes and leave the rest of the tree untouched. The following diagram illustrates the pointer rearrangement that must occur in order to perform these rotations.

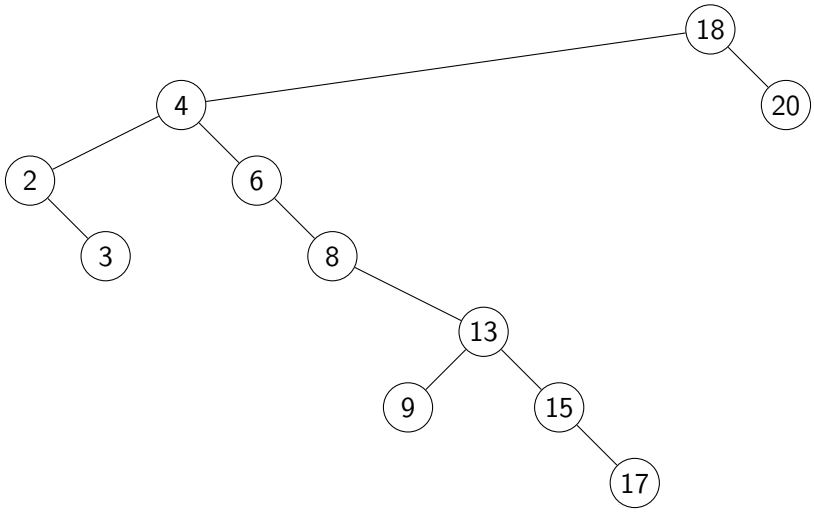


Note: Both methods take x as input, therefore the names of the variables in the code do not correspond one-to-one with the illustration without some relabeling.

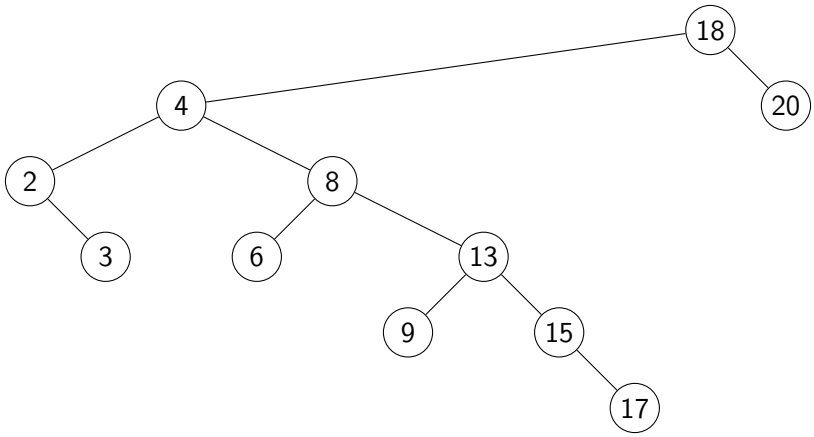
```
function RightRotate(T,x)
    y = x.left
    b = y.right
    Transplant(T,x,y)
    x.left = b
    if b ≠ NIL then
        b.parent = x
    end if
    y.right = x
    x.parent = y
end function
```

```
function LeftRotate(T,x)
    y = x.right
    b = y.left
    Transplant(T,x,y)
    x.right = b
    if b ≠ NIL then
        b.parent = x
    end if
    y.left = x
    x.parent = y
end function
```

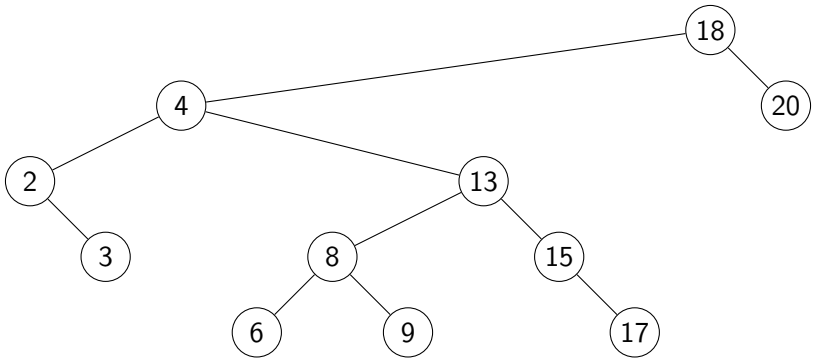
Apply a rotation and reduce the height of the tree



One rotation that reduces the height of the tree is a left rotation on 6.



We can further reduce the height by other rotations. We shall apply a left rotation on 8.



We leave it to the reader to try and reduce the height once more. It may take more than one rotation to accomplish this!